

南京大学

操作系统（2026 春）期中试卷

学号_____ 姓名_____ 专业_____ 总分_____

题号	一	二	三
得分			

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int pipe_fd[2];
    pipe(pipe_fd);
    int shared = 0;

    // 创建两个写者子进程 (fork)
    for (int i = 0; i < 2; i++) {
        if (fork() == 0) {
            close(pipe_fd[0]); // 关闭读端
            char buf[2048];
            // 将buf填充为字符 'A'+i (写者0填'A', 写者1填'B')
            memset(buf, 'A' + i, sizeof(buf));
            write(pipe_fd[1], buf, sizeof(buf));
            shared = i + 1;
            close(pipe_fd[1]); // 关闭写端
            printf("Writer %d: shared = %d\n", i, shared);
            exit(0);
        }
    }

    // 读者子进程 (vfork)
    if (vfork() == 0) {
        close(pipe_fd[1]); // 关闭写端
        char buffer[4096];
        int n;
        // 循环读取直到EOF
        while ((n = read(pipe_fd[0], buffer, sizeof(buffer))) > 0) {
            printf("Reader: read %d bytes, first byte = %c, shared = %d\n",
                n, buffer[0], shared);
        }
        shared = 100;
        _exit(0);
    }
}
```

```
// 父进程
close(pipe_fd[0]);
close(pipe_fd[1]);
wait(NULL); // 等待第一个子进程 (任意一个) 退出
wait(NULL); // 等待第二个子进程退出
wait(NULL); // 等待第三个子进程退出
printf("Parent: shared = %d\n", shared);
return 0;
```



一、问答题

阅读以上 C 程序（Linux 环境），回答后续三个问题。

- (1) 读者进程(即 vfork 创建的子进程)可能输出哪些（字节数，字节序列）？请列出所有可能情况并解释原因。
- (2) 分析该程序运行后是否会发生死锁？若会，请描述死锁产生的原因；若不会，请说明理由。
- (3) 程序中四个进程打印的 shared 变量的值分别是多少？请解释原因。

二、计算题

考虑一个采用基于优先级的抢占式时间片轮转调度算法的系统。调度规则如下：

- 每个进程有一个静态优先级，数值越小表示优先级越高。
- 就绪队列按优先级排序，同一优先级内的进程采用时间片轮转调度（时间片长度固定为 2 个时间单位）。
- 进程到达时，如果其优先级高于当前正在运行的进程，则立即抢占 CPU，被抢占的进程放回其所属优先级队列的末尾。
- 当进程的时间片用完但尚未完成时，它被放回当前优先级队列的末尾，并重新参与调度。
- 所有进程均为 CPU 密集型，不考虑 I/O 等待。



进程	到达时间	执行时间	优先级
P1	0	8	2
P2	1	4	1
P3	2	2	3
P4	3	6	2

要求：

- (1) 画出上述调度算法的时间线图（即从 0 时刻开始，每个时间片内运行的进程），并写出所有进程的执行顺序（例如：P2 → P1 → P4 → ...）。
- (2) 计算每个进程的响应时间（从到达时刻到首次获得 CPU 的时间间隔）以及平均响应时间。
- (3) 观察你得到的调度顺序和响应时间，你认为该调度算法是否公平？为什么？如果不公平，请提出一种改进思路，使得所有进程都能更公平地获得 CPU 时间，避免某个进程长时间得不到调度。请简要描述你的改进方法。



三、编程题

在一个机票预订系统中，共享剩余座位数 `seats`（初值为正）和当前票价 `price`。三类并发进程：查询进程可同时读取 `seats` 和 `price`；调价进程修改 `price`，与任何其他进程互斥；预订进程读取 `price`，并原子地执行“读 `seats` → 若大于零则 `seats--`”。预订进程之间互斥，与调价进程互斥，与查询进程的规则是：预订进程读 `seats` 时不阻塞查询，但修改 `seats` 时必须与查询互斥（即修改期间不允许查询读取）。不允许超售。请使用信号量（P/V 操作）实现，给出伪代码。

